

Règles d'intégration

Features & Composants Front-End

Guide complet pour penser, architecturer et intégrer des composants UI de qualité production.

I. Avant de coder — La phase de pensée

Questions obligatoires avant toute feature

- Est-ce que cette feature résout un vrai problème utilisateur ou c'est du "nice to have" interne ?
- Est-ce que ça existe déjà ailleurs dans le codebase sous une autre forme ?
- Si je retire cette feature demain, est-ce que quelqu'un le remarque ?
- Est-ce que cette feature peut vivre seule ou dépend-elle d'autres features pour avoir du sens ?
- Quel est le pire état possible de ce composant ? (vide, erreur, chargement, overflow de contenu)

Règles de scope

- Définir les états avant de définir le happy path : empty, loading, error, success, partial data.
- Définir qui possède la feature : équipe, page, contexte — jamais orpheline.
- Écrire en une phrase ce que le composant fait. Si la phrase contient "et", le composant fait trop de choses.

II. Architecture du composant

Responsabilité unique

- Un composant = une responsabilité. Séparation stricte entre logique, présentation, et données.
- Les composants "smart" (connectés aux données) ne font pas de rendu complexe.
- Les composants "dumb" (purement UI) ne savent pas d'où viennent leurs données.
- Jamais de fetch dans un composant de présentation.
- Jamais de style hardcodé dans un composant logique.

Règles de props

- Moins de 5 props = bon signe. Plus de 7 = le composant fait trop de choses, le découper.
- Toujours typer les props (TypeScript, PropTypes) — jamais de any.
- Les props optionnelles ont toujours une valeur par défaut.
- Éviter les props booléennes qui changent le comportement fondamental (isAdmin, isAlt, isV2) — préférer des variantes ou des composants distincts.
- Ne jamais passer un objet entier quand seuls 2 champs sont utilisés.

Règles de composition

- Préférer la composition à l'héritage : `<Component />`, plutôt que `<Component prop={...} />`.
- Les slots et children permettent d'étendre sans modifier.
- Un composant ne doit jamais connaître son parent.

III. Intégration dans la page / le layout

Positionnement

- Toujours penser en termes de flux naturel avant de penser en position absolue ou fixed.
- Le composant doit fonctionner dans un conteneur de 320px (mobile) comme dans 1440px (desktop).
- Ne jamais fixer une hauteur sur un conteneur de contenu dynamique — utiliser min-height.
- Les composants ne se positionnent pas eux-mêmes dans la page : c'est le layout parent qui s'en charge.
- Éviter les margin sur les composants réutilisables, préférer gap dans le parent.

Espacement

- Tout espacement est un multiple de 4 ou 8px — aucune valeur arbitraire.
- L'espacement interne (padding) appartient au composant.
- L'espacement externe (margin, gap) appartient au parent ou au layout.
- Plus deux éléments sont liés sémantiquement, plus ils sont proches visuellement (loi de proximité).

Z-index

- Définir une échelle z-index globale : base (0), elevated (10), dropdown (100), modal (200), toast (300), tooltip (400).
- Jamais de z-index arbitraire (z-index: 9999) — si ça arrive, c'est un problème de stacking context.
- Chaque nouveau layer de z-index doit être justifié.

IV. Gestion des états du composant

Les 7 états obligatoires

- Default : l'état normal avec des données réelles (pas du lorem ipsum).
- Loading / Skeleton : jamais un spinner seul si un skeleton est faisable.
- Empty : un vrai message utile — expliquer pourquoi et proposer une action.
- Error : message clair, action de récupération (retry, contact support).
- Disabled : toujours expliquer pourquoi c'est désactivé si ce n'est pas évident.
- Hover / Focus / Active : obligatoires pour tout élément interactif.
- Overflow : que se passe-t-il avec un titre de 200 caractères ? Une liste de 10 000 items ?

Feedback utilisateur

- Toute action utilisateur doit produire un retour en moins de 100ms (même juste un état visuel).
- Les états de loading bloquants (full page) sont le dernier recours — préférer optimistic UI ou skeleton inline.
- Les erreurs ne sont jamais silencieuses côté utilisateur.

V. Responsive et adaptabilité

Règles de base

- Mobile-first dans le CSS : les styles de base s'appliquent au mobile, les breakpoints élargissent.

- Ne jamais cacher du contenu important sur mobile — le réorganiser.
- Les breakpoints sont définis globalement (tokens), jamais dans le composant lui-même.
- Tester à 320px, 375px, 768px, 1024px, 1440px — ces tailles couvrent 95% des cas réels.
- Les images et médias ont toujours max-width: 100% et un ratio préservé.

Interactions tactiles

- Toute cible interactive fait au minimum 44x44px sur mobile.
- Espacer les éléments cliquables d'au moins 8px entre eux.
- Éviter les interactions hover-only — elles n'existent pas sur touch.

VI. Performance front

Chargement

- Tout composant lourd (tableau, carte, chart) est lazy-loadé si non visible above-the-fold.
- Les images ont toujours loading="lazy" sauf celles above-the-fold (LCP).
- Éviter les re-renders inutiles : mémoiser les composants purs et les callbacks coûteux.
- Ne jamais bloquer le thread principal avec du calcul synchrone — web workers ou découpage en chunks.

Bundle

- Vérifier l'impact bundle de chaque dépendance ajoutée (bundlephobia.com).
- Préférer les imports nommés aux imports default sur les grosses librairies (tree-shaking).
- Une feature non utilisée par 80% des users est un bon candidat au code splitting.

VII. Accessibilité (non négociable)

Règles fondamentales

- Tout élément interactif est atteignable au clavier (Tab, Enter, Escape, flèches).
- Les images décoratives ont alt="", les images informatives ont un alt descriptif.
- Les icônes sans texte ont un aria-label.
- Les modales trappent le focus et le restituent à l'élément déclencheur à la fermeture.
- Le ratio de contraste texte/fond est au minimum 4.5:1 (WCAG AA).
- Ne jamais utiliser la couleur seule pour transmettre une information — ajouter une icône ou un texte.
- Les formulaires ont des labels explicites liés (for / aria-labelledby), pas juste des placeholders.

VIII. Intégration dans un Design System

Règles DS

- Si le composant n'existe pas dans le DS : vérifier d'abord si un composant existant peut être composé.
- Si un nouveau composant est créé : il doit être généralisable (pas lié à un seul use case métier).
- Nommer les composants selon ce qu'ils sont, pas ce qu'ils font : pas .

- Documenter les cas d'usage, les cas limites, et les anti-patterns du composant.
- Chaque token modifié dans le DS impacte tous les composants — traiter comme un breaking change.

IX. Le test du "6 mois après"

Questions avant de merger

- Un développeur qui n'a pas participé peut-il comprendre ce composant en moins de 5 minutes ?
- Si les données changent de forme, combien d'endroits faut-il modifier ?
- Si le design change, est-ce que la logique survit ?
- Si la feature est retirée, le reste du code reste-t-il propre ?
- Si une réponse est non ou "ça dépend" → refactoriser avant de livrer.